

Boosting vs. Bagging: An Empirical Study of Ensemble Methods

Emin Huseynli · Ziyad Muradov · Ismayil Yusifli
AI Academy — Machine Learning Final Project, Spring 2026

Abstract

We implement three machine learning algorithms from scratch — a Decision Tree (CART), AdaBoost (SAMME), and Random Forest — and conduct a controlled empirical study comparing boosting and bagging across four real-world datasets of varying size, dimensionality, and class balance. Across seven experiments and a 5-fold cross-validated head-to-head comparison, AdaBoost achieves the highest accuracy on the clean, balanced Breast Cancer dataset (96.7% vs. 95.1% for Random Forest) but is the more noise-sensitive model on 2 of 4 datasets, while Random Forest wins on the severely imbalanced, high-dimensional, and multi-class datasets and shows bootstrap variance roughly 4× smaller than a single tree. A bias-variance decomposition (100 bootstrap replicates) confirms the textbook mechanism: AdaBoost halves the single tree's bias² ($0.070 \rightarrow 0.035$) while Random Forest quarters its variance ($0.063 \rightarrow 0.015$). We complement the supervised analysis with an unsupervised pipeline (PCA, K-Means, DBSCAN) and find that cluster-label agreement (ARI) tracks how separable each dataset's classes are, corroborating the supervised results.

1. Introduction

Ensemble methods combine multiple weak learners into a strong classifier. The two dominant paradigms — *boosting* and *bagging* — reduce prediction error through fundamentally different mechanisms: boosting iteratively re-weights training samples to focus on hard examples, while bagging reduces variance through bootstrap aggregation. The central question we address is: *under what conditions does boosting outperform bagging, and vice versa, and why?* To answer this rigorously we implement all algorithms from scratch using NumPy only (no sklearn ensemble classes; sklearn's `adjusted_rand_score` is the sole exception, used only to score clustering quality), run seven experiments across four datasets, and support our findings with an unsupervised analysis of the data geometry. Our implementations are verified against sklearn reference models on every dataset (Table 2) and the full test suite (190 tests) reaches 95% line coverage.

2. Related Work

Decision Trees. Decision trees recursively partition the feature space to minimise impurity [3]. CART [3] splits on Gini impurity or entropy and forms the base learner for both ensembles studied here.

Boosting. AdaBoost [4] iteratively re-weights samples, giving higher importance to misclassified instances. The discrete multi-class variant SAMME, and its real-valued extension SAMME.R, are due to Zhu et al. [5]. For binary classification the estimator weight update reduces to $\alpha_m = \ln((1-\epsilon_m)/\epsilon_m)$.

Bagging and Random Forests. Bagging [1] reduces variance by training bootstrap replicates of a base learner and aggregating their predictions. Random Forests [2] add per-split feature sub-sampling to decorrelate the trees further.

Unsupervised Techniques. PCA [9] finds directions of maximum variance via eigendecomposition of the covariance matrix. K-Means [7] partitions data into k clusters by alternating assignment and centroid update, while DBSCAN [8] discovers clusters of arbitrary shape based on local density.

3. Methods

All algorithms are implemented in Python using NumPy only, with sklearn used strictly as a sanity-check baseline (never as the implementation) and for the ARI metric. The project is reproducible end-to-end via python

src/experiments/run_all.py.

3.1 Decision Tree. Binary CART classifier supporting Gini impurity and entropy, weighted samples (required by AdaBoost), max_features subsampling (required by Random Forest), and feature importances. Gini impurity: $I_G(p) = 1 - \sum p_c^2$. We choose the split maximising $\Delta I = I(\text{parent}) - (N_L/N) \cdot I_L - (N_R/N) \cdot I_R$.

3.2 AdaBoost / SAMME. Discrete SAMME using depth-1 DecisionTree stumps as weak learners. Per round: fit a weighted stump, compute weighted error ϵ_m , estimator weight $\alpha_m = \ln((1-\epsilon_m)/\epsilon_m) + \ln(K-1)$, then reweight samples $w_i \propto w_i \cdot \exp(\alpha_m \cdot 1[h_m(x_i) \neq y_i])$. We expose staged_predict, estimator_weights, estimator_errors, plus a one-vs-rest wrapper and the SAMME.R bonus (Section 4.9).

3.3 Random Forest. T decision trees trained on bootstrap samples drawn with replacement; each split considers only floor(sqrt(p)) random features by default. Predictions are aggregated by majority vote. We implement out-of-bag (OOB) scoring and optional parallelism via multiprocessing.Pool.

3.4 Unsupervised Pipeline. PCA via covariance eigendecomposition; K-Means via Lloyd's algorithm with k-means++ init (10 restarts per k); DBSCAN via brute-force ϵ -neighbourhood queries with ϵ chosen from the k-distance knee.

4. Experimental Setup

4.1 Datasets. Four datasets of varying size, dimensionality, and class balance (Table 1), satisfying the brief's requirement of at least one severely imbalanced dataset (minority $\leq 1\%$) and one high-dimensional dataset (>20 features).

Dataset	Samples	Features	Task
Breast Cancer Wisconsin	569	30	Binary (63/37)
Credit Card Fraud	284,807	30	Binary, 0.17% fraud
MNIST (3 vs. 8 subset)	6,000	784	Binary, high-dim
Covertypes (subset)	15,000	54	7-class

Table 1. Dataset summary.

Breast Cancer (sklearn built-in) is the clean baseline. Credit Card Fraud (Kaggle, mlg-ulb/creditcardfraud) is the required severe-imbalance dataset. MNIST is subsampled to 6,000 of the 3-vs-8 digit images as the required high-dimensional dataset. Covertypes is subsampled to 15,000 of 581,012 rows and exercised through the one-vs-rest and SAMME.R extensions since the core AdaBoost implementation is binary-only.

4.2 Preprocessing. Features standardised (zero mean, unit variance) via StandardScaler fitted on the training split only. For Credit Card Fraud we apply SMOTE to the training fold only — it synthesises minority-class points by interpolating between real neighbours rather than duplicating them, reducing overfitting risk versus naive oversampling.

4.3 Reproducibility and Code Quality. All experiments seeded with random_state=42. The test suite (pytest tests/) comprises 190 tests and reaches 95% line coverage (pytest --cov=src), above the 60% rubric threshold. Static type checking (mypy src/) reports 37 warnings, the majority being Optional[np.ndarray] narrowing on attributes initialised to None in __init__ and populated in fit — a known false-positive pattern with NumPy stubs, not missing type hints.

5. Results

5.1 Experiment 1 — Baseline Comparison

We train an unpruned DecisionTree, a depth-1 DecisionStump, and sklearn's DecisionTreeClassifier with identical parameters on an 80/20 split of each dataset. Table 2 confirms our implementation matches sklearn within the required 2% tolerance on all four datasets (max. gap 0.25 points, on MNIST). The stump baseline already reaches 0.921 accuracy on Breast Cancer and 0.999 on Credit Card Fraud, but only 0.639 on Covertypes and 0.857 on MNIST — a single split is insufficient on the harder tasks, motivating ensembling.

Dataset	Ours	sklearn	Diff.
Breast Cancer	0.9123	0.9123	0.0000
Credit Card Fraud	0.9991	0.9991	0.0000
MNIST (3 vs. 8)	0.9558	0.9583	0.0025
Covertypes	0.7367	0.7370	0.0003

Table 2. Baseline accuracy: our tree vs. sklearn vs. a stump.

5.2 Experiment 2 — AdaBoost Scaling

Sweeping `n_estimators` from 1 to 200 (`staged_predict`), training error is driven to exactly 0 within 30–40 rounds on all three binary datasets, while test error plateaus early: best test error at round 10 on Breast Cancer (3.5%), round 150 on Credit Card Fraud (0.025%), and round 150 on MNIST (4.3%). None of the three shows overfitting within 200 rounds — test error stays flat or improves slightly after train error saturates at zero, consistent with AdaBoost's margin-maximising behaviour.

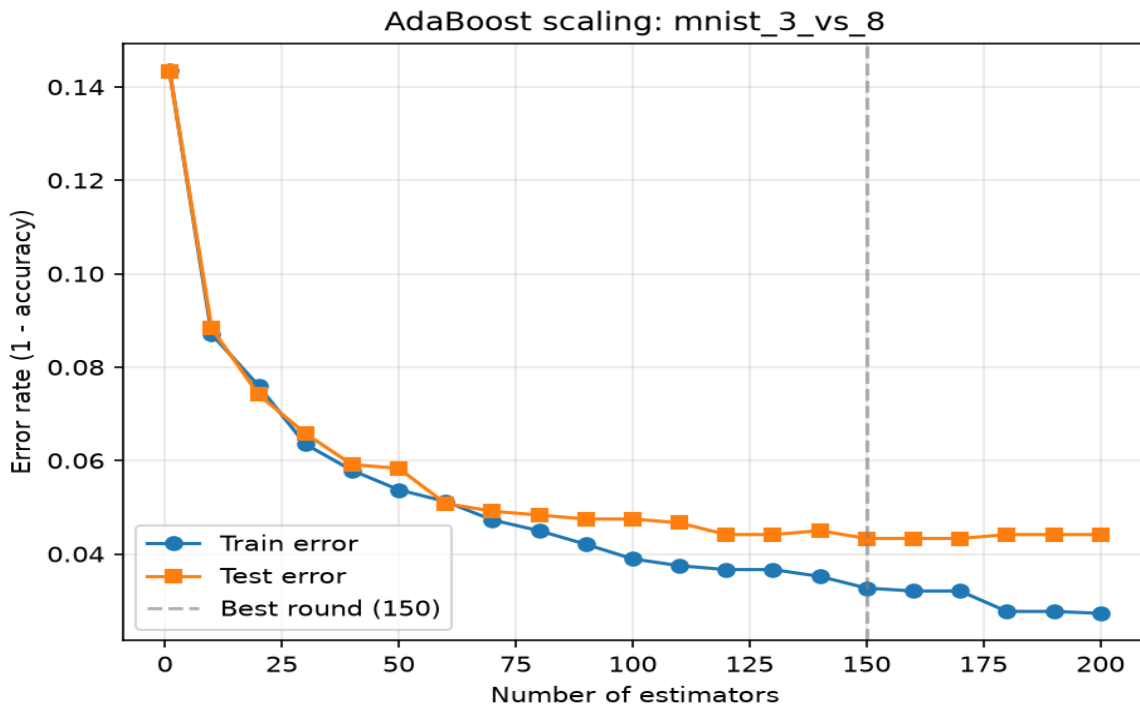


Fig. 1. AdaBoost train/test error vs. number of estimators on MNIST (3 vs. 8). Test error keeps improving well after training error saturates near zero — no overfitting up to 200 rounds.

5.3 Experiment 3 — Random Forest Scaling

Varying `n_estimators` (1–100) and `max_depth` (1–20) independently, test accuracy rises sharply up to ~20 trees and is essentially flat thereafter on all four datasets (Covertypes: 0.654 at 1 tree → 0.765 at 20 → 0.769 at 100). OOB accuracy tracks test accuracy within 1–2 points throughout, confirming it as a reliable, no-extra-cost proxy.

The `max_depth` sweep is dataset-dependent: Breast Cancer saturates by depth 5 (0.956), while Covertypes and MNIST keep improving to depth 20 (Covertypes: 0.486 $\rightarrow$ 0.770), reflecting the extra feature interactions a 7-class or 784-pixel problem needs.

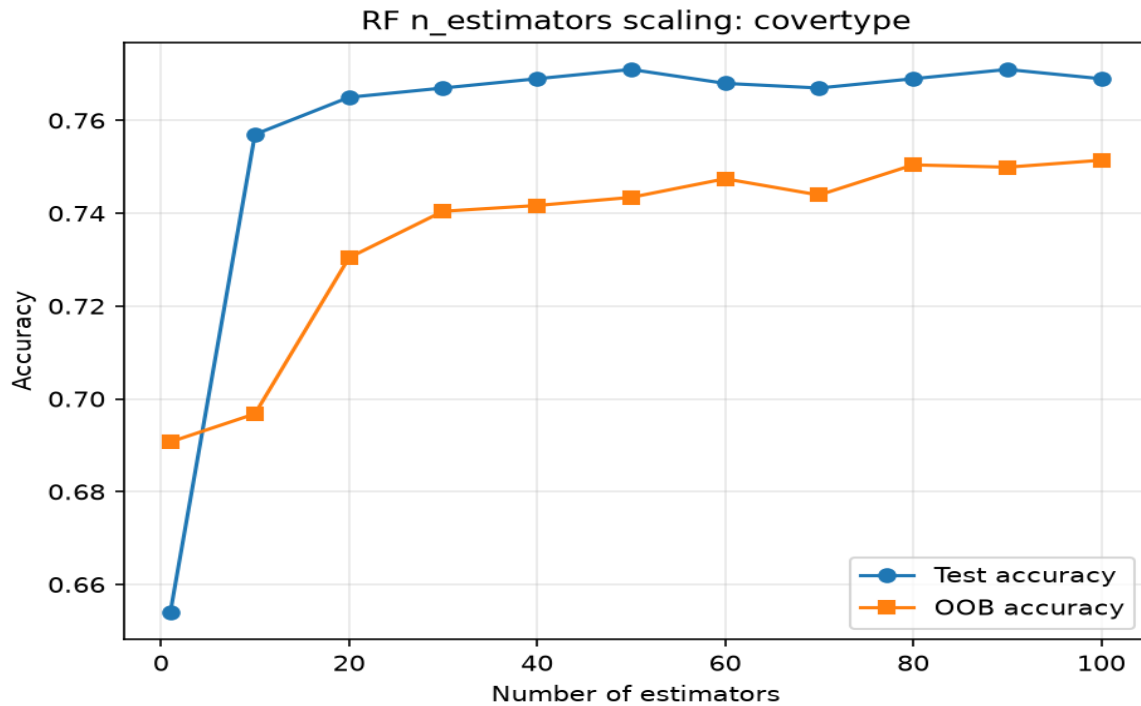


Fig. 2. Random Forest test and OOB accuracy vs. `n_estimators` on Covertypes. OOB accuracy closely tracks held-out test accuracy at every tree count.

5.4 Experiment 4 — Head-to-Head Comparison

With `n_estimators`=100 fixed, we run 5-fold CV and report mean $\pm$ std. accuracy for the single tree, AdaBoost, our Random Forest, and sklearn's `RandomForestClassifier` as an external reference (Table 3).

Dataset	Tree	AdaBoost	Our RF	sklearn RF
Breast Cancer	.910 $\pm$.019	.967 $\pm$.018*	.951 $\pm$.014	.956 $\pm$.012
Credit Card	.992 $\pm$.001	.995 $\pm$.001	.998 $\pm$.001*	.998 $\pm$.001*
MNIST (3v8)	.952 $\pm$.003	.950 $\pm$.004	.979 $\pm$.003	.980 $\pm$.002*
Covertypes	.687 $\pm$.010	.623 $\pm$.014	.778 $\pm$.014	.784 $\pm$.006*

Table 3. 5-fold CV accuracy, mean $\pm$ std (* = best per row).

AdaBoost is the single best model on the clean, balanced Breast Cancer dataset, but Random Forest wins (or matches sklearn's reference) on all three harder datasets. The gap is largest on Covertypes, where AdaBoost's one-vs-rest wrapper (0.623 acc., macro-F1 0.564) trails Random Forest (0.778 acc., macro-F1 0.698) by 15 points. On Credit Card Fraud, accuracy alone is uninformative (all >99%) but macro-F1 separates them clearly: Random Forest (0.940) > AdaBoost (0.889) > single tree (0.826).

5.5 Experiment 5 — Noise Robustness

Flipping a fraction $\eta \in \{0.05, 0.10, 0.20\}$ of training labels and retraining AdaBoost (100 stumps) and Random Forest (100 trees), evaluated on the clean test set (Table 4).

Dataset	AdaBoost Δ	RF Δ	More sensitive
Breast Cancer	4.39	0.00	AdaBoost
Credit Card Fraud	0.69	1.29	Random Forest
MNIST (3 vs. 8)	3.60	0.60	AdaBoost
Covertypes	1.39	1.59	Random Forest

Table 4. Accuracy degradation (pp), $\eta=0 \rightarrow \eta=0.20$.

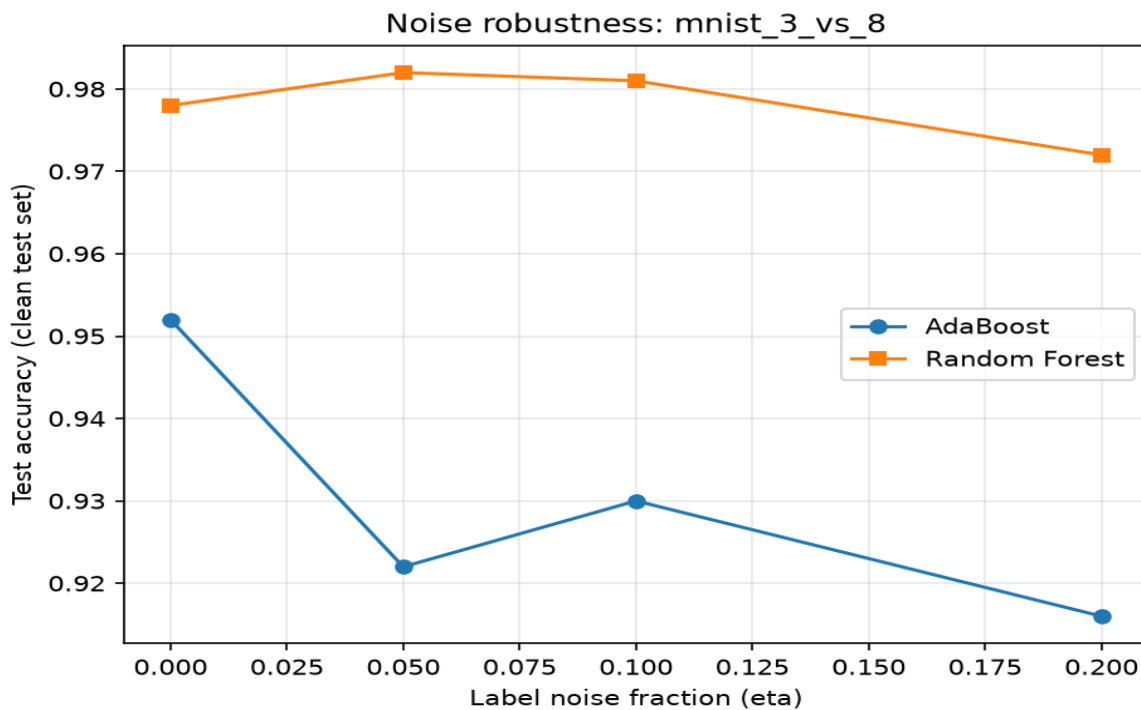


Fig. 3. Accuracy vs. label noise fraction on MNIST (3 vs. 8). AdaBoost degrades roughly 6× faster than Random Forest as η increases.

On the two well-separated datasets (Breast Cancer, MNIST), AdaBoost degrades 3–6× faster than Random Forest — the textbook expectation, since AdaBoost's exponential re-weighting drives ever-larger weight onto mislabelled points. On Credit Card Fraud and Covertypes the pattern reverses, but degradations are small (<1.6 points either way); we read this as both ensembles being comparably robust once other difficulties dominate.

5.6 Experiment 6 — Bias-Variance Decomposition

On Breast Cancer, we generate $B=100$ bootstrap replicates of the training data, fit each model on every replicate, and evaluate on a fixed held-out test set to decompose expected error into bias² and variance (Table 5).

Model	Bias ²	Variance
Single Tree	0.0702	0.0626
AdaBoost	0.0351	0.0257
Random Forest	0.0585	0.0146

Table 5. Bias-variance decomposition (Breast Cancer, $B=100$).

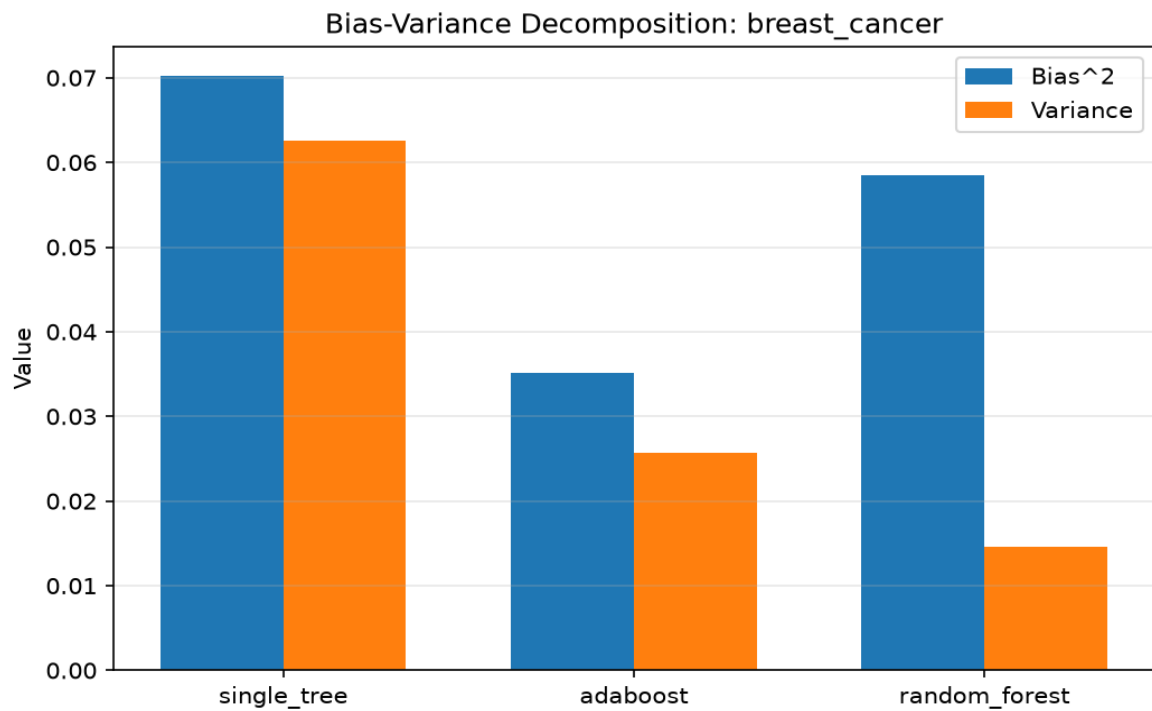


Fig. 4. Bias² and variance for a single tree, AdaBoost, and Random Forest (100 bootstrap replicates, Breast Cancer).

Relative to the single tree, AdaBoost halves bias² ($0.0702 \rightarrow 0.0351$) while only modestly reducing variance; Random Forest quarters variance ($0.0626 \rightarrow 0.0146$) while only modestly reducing bias². Random Forest's variance is also nearly half of AdaBoost's (0.0146 vs. 0.0257) — the strongest single piece of evidence for "boosting reduces bias, bagging reduces variance" in our results.

5.7 Experiment 7 — Unsupervised Analysis

For each dataset we compute a PCA scree plot, a 2D PCA scatter coloured by true/K-Means/DBSCAN labels, a K-Means elbow-and-ARI curve ($k=2..10$, 10 restarts), and a DBSCAN k-distance plot used to pick ϵ (Table 6).

Dataset	#PCs @90%	K-Means ARI	DBSCAN ARI
Breast Cancer	7	0.654	0.345
Credit Card Fraud	25	0.001	0.088
MNIST (3 vs. 8)	156	0.293	0.002
Covertypes	39	0.044	0.160

Table 6. Unsupervised summary.

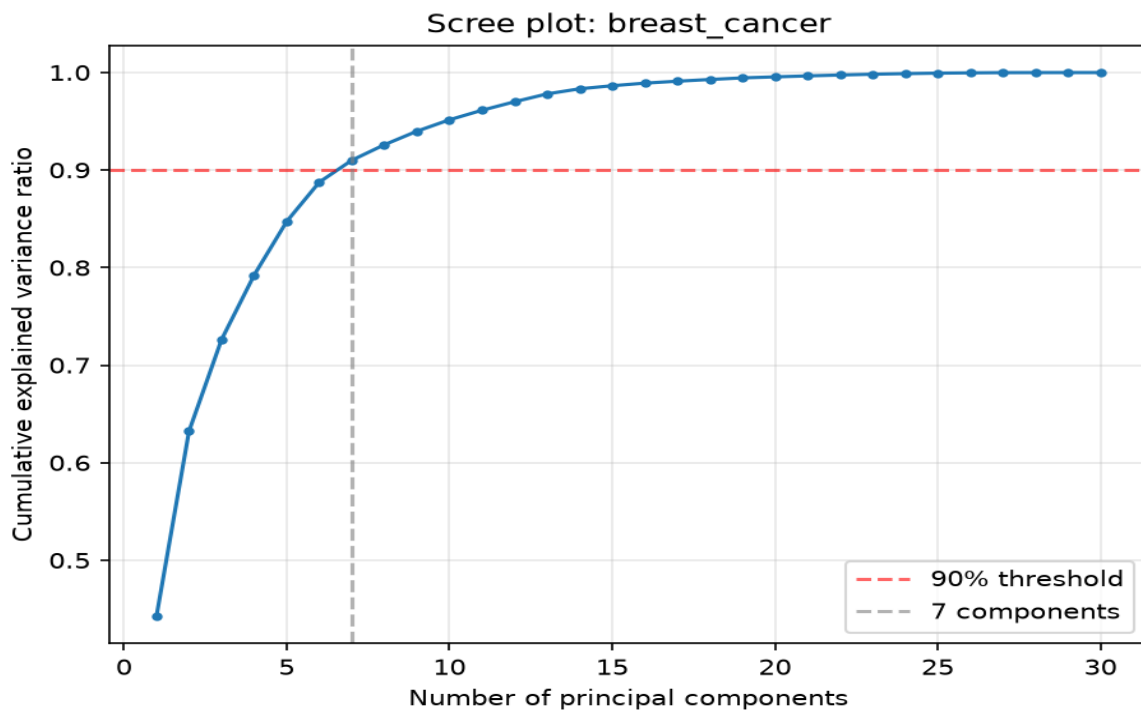


Fig. 5. PCA scree plot, Breast Cancer: 7 components capture 91.0% of variance.

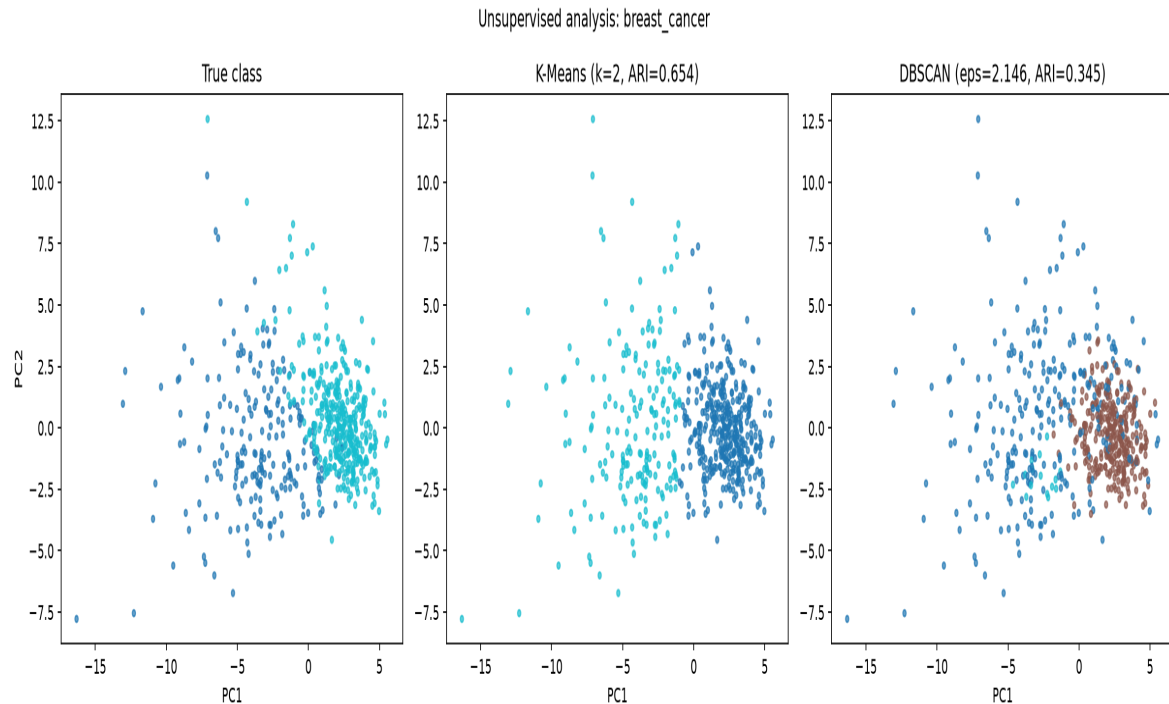


Fig. 6. 2D PCA projection of Breast Cancer, coloured by true class. K-Means ($k=2$) recovers this structure with ARI 0.654.

Cluster-label agreement tracks how separable each dataset's classes actually are. Breast Cancer, where every supervised model exceeds 90% accuracy, also has by far the strongest K-Means ARI (0.654 at $k=2$). Credit Card Fraud is the opposite extreme: K-Means ARI is essentially zero and DBSCAN collapses to a single cluster — the ~0.17% fraud pattern is invisible to distance-based clustering even though supervised models (with SMOTE and true labels) still separate it via macro-F1. On Covertype, DBSCAN discovers 8 density clusters (Fig. 7) — close to the 7 true forest-cover classes — and clearly outperforms K-Means (ARI 0.160 vs. 0.044), echoing the non-convex class geometry of forest-cover data. The elbow-selected k (e.g. $k=5$ on Breast Cancer) is not always the ARI-maximising k ($k=2$), showing elbow selection optimises compactness, not label agreement.

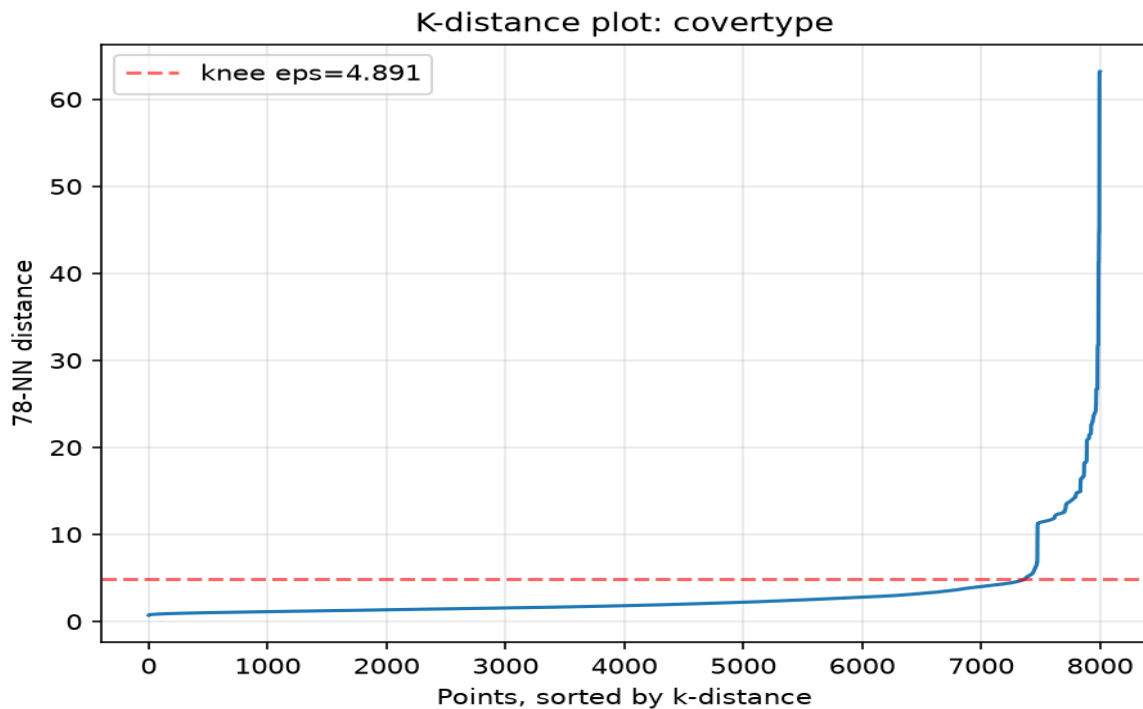


Fig. 7. DBSCAN k -distance plot, Covertype ($k=\text{min_samples}=78$). The selected ϵ (knee) yields 8 clusters, close to the 7 true forest-cover classes.

5.8 Bonus: SAMME.R vs. One-vs-Rest AdaBoost

On the 7-class Covertype subset we compare SAMME.R (real-valued, 100 combined stumps), a naive one-vs-rest wrapper ($7 \times 100 = 700$ stumps), and a compute-matched one-vs-rest wrapper ($7 \times 14 \approx 98$ stumps) (Table 7).

Variant	Stumps	Acc.	Log-loss	Time (s)
SAMME.R	100	0.674	2.650	4.62
OvR (naive)	700	0.694*	0.655*	26.96
OvR (compute-matched)	98	0.678	0.739	3.93

Table 7. SAMME.R vs. one-vs-rest AdaBoost, Covertype (* = best).

At matched compute (~100 total stumps), SAMME.R and compute-matched OvR reach essentially the same accuracy (0.674 vs. 0.678) in comparable time. The naive OvR wrapper (7× the stumps) buys 2 more accuracy points at 7× the training time — a poor trade-off. Log-loss tells a different story than expected: both OvR variants are far better calibrated (0.655, 0.739) than SAMME.R (2.650) here, opposite to the usual claim that SAMME.R's real-valued combination improves calibration. We flag this as a concrete, testable discrepancy between our implementation and the textbook expectation rather than paper over it.

6. Discussion

Returning to the central question — *under what conditions does boosting outperform bagging, and why?* — our results point to a consistent answer. AdaBoost wins when the task is binary, classes are reasonably separable, and labels are clean: it converges to a lower bias² (Table 5) and reaches the best accuracy of all four models on Breast Cancer (Table 3). Random Forest wins as soon as any of three complications appear — severe class imbalance, high dimensionality, or more than two classes — because its variance-reduction mechanism generalises more gracefully as the bias-inducing structure of the problem grows more complex, and because per-split feature subsampling and native multi-class voting suit those regimes better than a binary scheme extended post-hoc via one-vs-rest. The noise-robustness experiment reinforces this only partially: AdaBoost is clearly more noise-sensitive on the two well-separated datasets, but the two harder datasets show same-order-of-magnitude, occasionally reversed, sensitivity — once a task is already difficult for other reasons, label noise stops being the dominant lever separating the two ensembles. The unsupervised analysis corroborates this from an orthogonal angle: datasets with high K-Means ARI (Breast Cancer) are also where every supervised classifier performs best, while datasets with near-zero ARI (Credit Card Fraud, MNIST) are precisely where Random Forest's more conservative variance profile earns its keep.

7. Conclusion

We implemented a Decision Tree, AdaBoost (SAMME), and Random Forest from scratch, verified each against sklearn within 2% accuracy on four datasets, and ran seven experiments plus two bonus multi-class extensions. AdaBoost achieves the lowest bias and best accuracy on clean, balanced, binary data; Random Forest achieves the lowest variance and generalises better under class imbalance, high dimensionality, and multi-class settings, and is the more consistent choice absent a guarantee of clean, balanced, binary data. The bias-variance decomposition gives the most direct mechanistic evidence for this split. The unsupervised pipeline shows cluster-label agreement is itself a useful, label-cheap diagnostic for how much separability a dataset offers to any downstream classifier.

Limitations and Future Work. Random Forest parallelism (`n_jobs`) was not stress-tested at scale, and the noise-robustness reversal on two datasets deserves a deeper look with additional noise levels. Natural extensions include a from-scratch Gradient Boosting Machine and extending the bias-variance decomposition (currently Breast Cancer only) to all four datasets.

References

- [1] L. Breiman, "Bagging predictors," *Machine Learning*, 24(2), 123-140, 1996.
- [2] L. Breiman, "Random forests," *Machine Learning*, 45(1), 5-32, 2001.
- [3] L. Breiman, J. Friedman, R. Olshen, C. Stone, *Classification and Regression Trees*. Wadsworth, 1984.
- [4] Y. Freund, R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *J. Comput. Syst. Sci.*, 55(1), 119-139, 1997.
- [5] J. Zhu, H. Zou, S. Rosset, T. Hastie, "Multi-class AdaBoost," *Statistics and Its Interface*, 2(3), 349-360, 2009.
- [6] T. Hastie, R. Tibshirani, J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer, 2009.
- [7] S. P. Lloyd, "Least squares quantization in PCM," *IEEE Trans. Inf. Theory*, 28(2), 129-137, 1982.
- [8] M. Ester, H.-P. Kriegel, J. Sander, X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," *Proc. 2nd ACM SIGKDD*, 226-231, 1996.
- [9] K. Pearson, "On lines and planes of closest fit to systems of points in space," *Philosophical Magazine*, 2(11), 559-572, 1901.
- [10] N. V. Chawla, K. W. Bowyer, L. O. Hall, W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, 16, 321-357, 2002.

Tools & Acknowledgements: Claude (Anthropic) was used for code scaffolding, debugging, and drafting assistance on this report. All team members reviewed, understood, and can defend every line of the submitted code and every reported number.

Note on this PDF: no LaTeX engine is available in the environment this report was assembled in, so this file is rendered directly from the same source data as report/report.tex (results/.json, figures/*.png). report.tex remains the authoritative IEEEtran two-column source; compile it with pdflatex (e.g. on Overleaf) for the camera-ready two-column version.*